

INFO GIVEN BY SIR

5. Use the road network digital data of IIT Bombay campus and track the movement of Autos and service vehicles in the campus. What are the pros and cons?

Submission:

Email: durbha.courses@gmail.com

Subject line: GNR605: Course

Project, 2025 <roll no.>

•

Project presentations are scheduled on Nov 25th at 11:00 am.

Deliverables

- ❖ Documented algorithm with flowchart
- ❖ Pseudo-code indicating the computational flow, data structures
- ❖ Sample data used
- ❖ Source code
- ❖ Compiled binaries
- ❖ Results
- ❖ Powerpoint presentation explaining the concept, algorithm, implementation and results as well as brief 10 minutes classroom presentation
- ❖ The assignments can be done using any of the programming languages

SIR'S EXPECTATION

1. useful for students/faculty : able to track the auto location

2. useful for security : when outside auto come inside, they will use the app to register, them self and then count down timer for them will be started and if they don't deregister them self and inside the campus, they will be placed in some special list.
{but don't think people will logout , as they are outsiders, and also not necessary they will log in.}

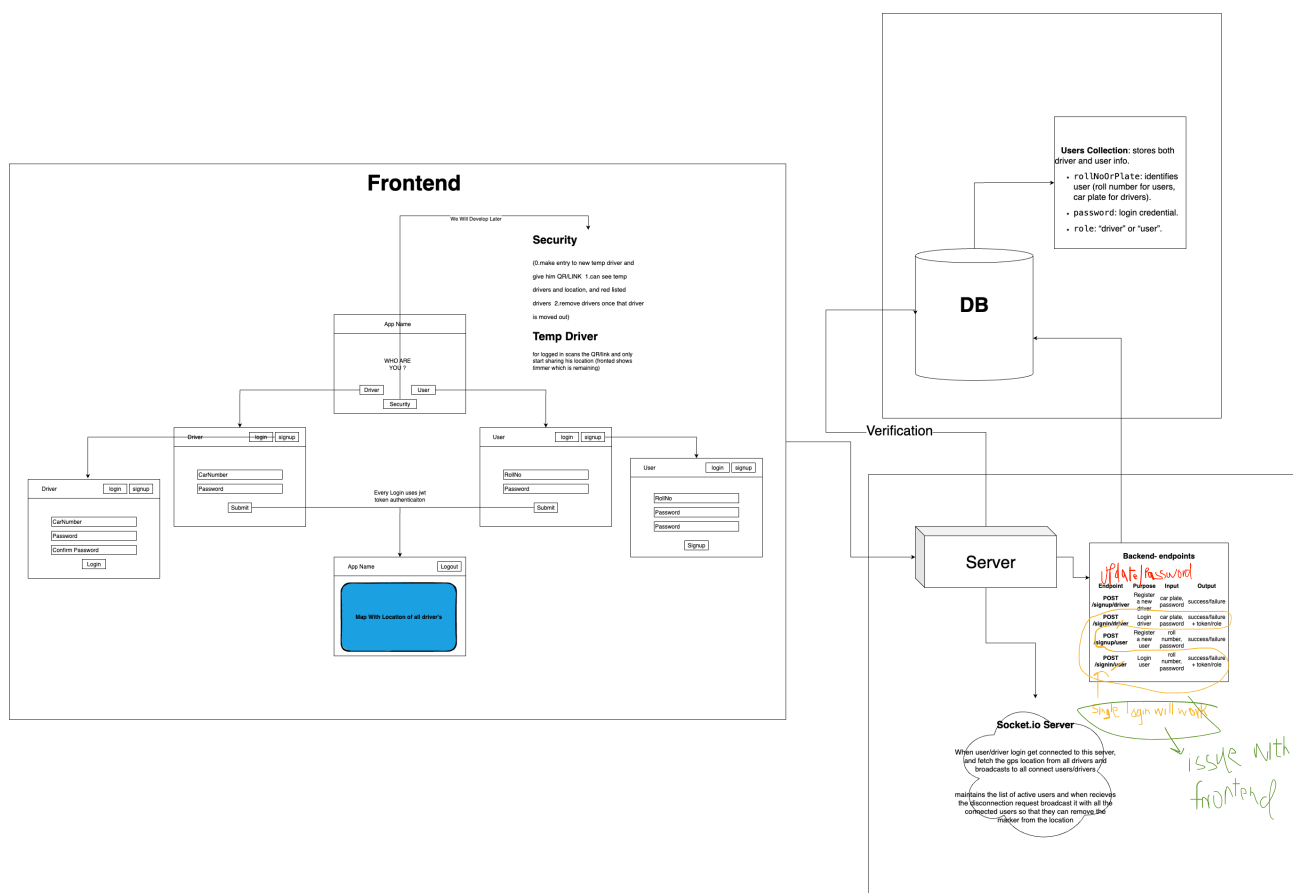
{first we will focus on 1. Then after that if time remains we will consider working on 2. , as stated in problem statement, only tracking part, but sir stated that in lecture, so even if not implemented should not be an issue}

Core Idea:

- Each **auto/service vehicle** has a mobile app (or web app) that sends its **GPS location** periodically to the server.
- A **central dashboard (web app)** shows live movement of these vehicles on the **campus map** using the IITB road network data.
- Users (students/faculty) can see where autos are in real time — like Ola/Uber mini version but restricted to campus.

Development Part:

Diagram :



Tech Stack: (Be Ready Many new terms you will see)

1. **Frontend:** React + **Leaflet.js** for maps (opensourced with many resources)

{

Driver Dashboard

- Uses browser **Geolocation API** to get GPS.
- Sends coordinates to backend via Socket.IO (emit("updateLocation")).

User Dashboard

- Connects to backend via Socket.IO (on("locationUpdate")).
- Maintains a list of active vehicles.
- Updates **Leaflet.js markers** when new coordinates are received.
- Leaflet handles the **visual map**; markers move as coordinates update.

}

{

 broadcasted location of drivers by server is recieved through socket.io will be shown by leaflet on each users frontend

}

2. **Backend:** Node.js + Express

{

 Your **backend** has two main responsibilities:

1. **Login/authentication** → verify driver/user credentials.
2. **Real-time location management** → receive driver GPS and broadcast to all users.

}

3. **Real-time updates:** **Socket.IO** (is library build on top of WebSocket Protocol)

{

 how it works?

Real-Time Communication (Socket.IO)

- **Socket.IO** enables **instant two-way communication** between frontend (browser) and backend (server).
- Workflow:
 1. **Driver logs in** → browser connects to Socket.IO server.

2. **Driver GPS:** browser continuously sends coordinates to server.
3. **Server:** receives driver location → immediately broadcasts it to all connected **users**.
4. **User dashboard:** receives live coordinates → updates the vehicle marker on the map.

}

4. **Database:** MongoDB (store vehicle info, and user data)

5. **Deployment / DevOps:** Render / Railway: Host backend (Express server).

Vercel / Netlify: Host frontend (React app).

MongoDB Atlas: Cloud database storage.

Environment Variables: Store sensitive keys (JWT secret, DB URI)

Flow Algo: (NOT Proper)

1 Login

[Driver/User] → POST /login → Backend → MongoDB (verify credentials) → Success/Failure → Frontend

2 Real-Time Location (Driver)

[Driver Dashboard] → Browser Geolocation → Socket.IO emit → Backend Server → Socket.IO broadcast → All connected Users

3 Show on Map (User)

[User Dashboard] → Socket.IO on('locationUpdate') → Update Marker on Leaflet Map

{how to handle when user is disconnected, but still the driver data is with me?

->

Remove / Mark driver as offline (better)

Steps:

1. Track active drivers on the backend

- Maintain a list or map: { `socketId: plate` } or { `plate: lastSeenTime` }.

2. Listen for disconnect events

- Socket.IO triggers `socket.on("disconnect")` whenever a driver disconnects.

3. **Broadcast driver offline**

- Notify all users that the driver is now offline:

```
io.emit("driverOffline", { plate: "MH01AB1234" })
```

4. **Update frontend map**

- Remove the marker or mark it as inactive.

```
}
```